PROJECT DOCUMENTATION

AI Interview Agent

An adaptive, resume-aware technical interview system
powered by a multi-agent AI architecture.

Stack: FastAPI · React · Anthropic Claude · **Model:** claude-sonnet-4-5 · **Version:** 1.0.0
Repository: github.com/Thanuja0911/interview-agent

Contents

1. Overview
2. Key Features
3. Architecture
4. Tech Stack
5. Repository Structure
6. Getting Started
7. Running an Interview
8. API Reference
9. The Adaptive Interview Loop
0. Agents & Prompts
1. Data Models
2. Testing
3. Design Notes & Trade-offs

1. Overview

The **AI Interview Agent** conducts personalized technical interviews driven by a candidate's resume. It evaluates each answer in real time using the *LLM-as-judge* pattern and adapts question difficulty dynamically based on rolling performance. The system is built as two independently run services: a Python/FastAPI backend that owns all AI orchestration and session state, and a React single-page frontend that visualizes the live interview.

Unlike a single-prompt chatbot, the system uses **specialized agents with distinct roles and temperatures** — a deterministic evaluator, a creative interviewer, and a holistic scorecard generator — composed deliberately into an adaptive loop.

2. Key Features

Multi-agent design with distinct roles

Two specialized Claude agents run per interview turn. The **EvaluatorAgent** runs at temperature 0 for consistent, deterministic judgment. The **InterviewerAgent** runs at temperature 0.7 for varied, non-repetitive question generation. Each has a purpose-built system prompt with explicit rubrics and behavioral rules.

Adaptive personalization from real signal

A rolling average score is recalculated after every answer and mapped to a difficulty level using concrete thresholds — above 7 escalates to hard, below 4 softens to easy. This changes the interviewer's prompt context, which changes its tone, question depth, and topic selection.

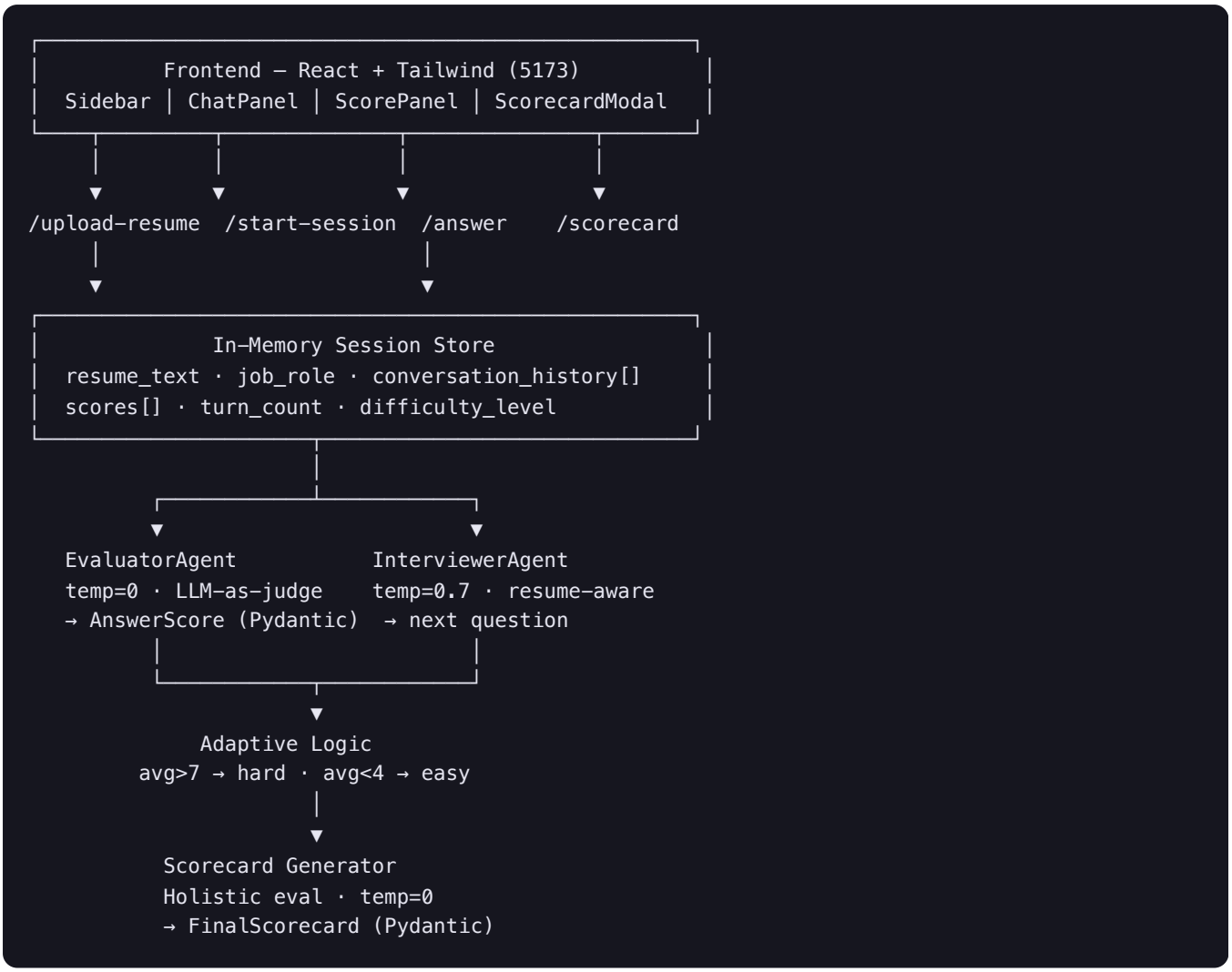
LLM-as-judge evaluation made visible

The evaluator returns a structured `AnswerScore` object validated by Pydantic before it reaches the frontend. The UI surfaces this judgment live — score ring, depth badge, coaching feedback, and follow-up detection — making the evaluation pipeline transparent.

Resume-aware context injection

The full parsed resume is injected into every question-generation prompt, grounding questions in the candidate's actual projects, technologies, and claims.

3. Architecture



The frontend never talks to Claude directly. Every AI call is orchestrated by the backend, which also owns all session state. The frontend is a pure presentation layer that renders scores, difficulty changes, and the final report.

4. Tech Stack

Layer	Technology	Version
Frontend	React + Vite	React 19
Styling	Tailwind CSS	v3
Icons	lucide-react	—
Backend	FastAPI	0.136+
Server	Uvicorn	0.46+
AI Provider	Anthropic Claude	claude-sonnet-4-5
Resume Parsing	pdfplumber	0.11+
Validation	Pydantic	v2

5. Repository Structure

```
interview-agent/
├── backend/
│   ├── main.py                # FastAPI app — endpoints + adaptive interview loop
│   ├── agents.py              # EvaluatorAgent, InterviewerAgent, scorecard generator
│   ├── session_store.py       # In-memory session dict + difficulty logic
│   ├── resume_parser.py       # PDF extraction + text cleaning (pdfplumber)
│   ├── schemas.py             # Pydantic models (AnswerScore, FinalScorecard)
│   ├── test_agent_loop.py     # Live terminal script (hits real Anthropic API)
│   ├── test_resume_parser.py  # Unit tests (mocked pdfplumber, no API key)
│   ├── requirements.txt       # Pinned Python dependencies
│   └── .env.example           # ANTHROPIC_API_KEY placeholder
└── frontend/
    ├── src/
    │   ├── api.js             # Central API service — all backend calls
    │   ├── App.jsx            # Top-level state + interview orchestration
    │   └── components/
    │       ├── Sidebar.jsx         # Resume upload + role configuration
    │       ├── ChatPanel.jsx       # Conversation + difficulty adaptation UI
    │       ├── ScorePanel.jsx      # Live score ring, depth, coaching
    │       └── ScorecardModal.jsx  # Final report + JSON export
    ├── package.json
    └── tailwind.config.js
```

6. Getting Started

Prerequisites

- Python 3.10+
- Node.js 18+

- An Anthropic API key (console.anthropic.com)

Backend

```
cd backend
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env              # then add your ANTHROPIC_API_KEY
uvicorn main:app --reload --port 8000
```

Backend runs at `http://localhost:8000`; interactive API docs at `http://localhost:8000/docs`.

Frontend

```
cd frontend
npm install
npm run dev
```

Frontend runs at `http://localhost:5173`.

> **Ports are coupled.** The frontend hardcodes the backend URL in `src/api.js` (`http://localhost:8000`), and the backend CORS policy in `main.py` allows only `http://localhost:5173`. Changing a port in one place without the other breaks the connection.

Environment Variables

Variable	Where	Purpose
`ANTHROPIC_API_KEY`	`backend/.env`	Authenticates all Claude calls. Required.

The client is constructed at import time in `agents.py`; without a key, every agent call fails. The model is pinned to `claude-sonnet-4-5` — never use a "latest" alias in production.

7. Running an Interview

1. Open `http://localhost:5173`.
2. Upload a PDF resume in the left sidebar.
3. Enter a target job role and optional company context.
4. Click **Start Interview** — the first question is generated immediately.
5. Answer in the chat panel; watch the score ring and difficulty badge update live.
6. Click **Finish Interview** after 2+ turns (5+ recommended) to generate the scorecard.
7. Click **Export Report** to download the full interview artifact as JSON.

8. API Reference

POST /upload-resume

Upload a PDF resume and initialize a session. `multipart/form-data`

Request: field `file` (PDF only).

```
{
  "session_id": "uuid-string",
  "resume_preview": "First 300 characters of parsed resume...",
  "character_count": 2847
}
```

POST /start-session

Attach a job role to a session and generate the first question.

```
{
  "session_id": "uuid-string",
  "job_role": "Senior Frontend Engineer",
  "company_context": "Fast-growing fintech startup"
}
```

Response:

```
{
  "status": "session_ready",
  "session_id": "uuid-string",
  "job_role": "Senior Frontend Engineer",
  "first_question": "You built a React weather app and ran into CORS errors...",
  "difficulty_level": "medium"
}
```

POST /answer

Submit a candidate answer. Runs the full adaptive interview loop.

```
{
  "session_id": "uuid-string",
  "current_question": "Walk me through how you fixed the CORS error...",
  "answer": "I routed the request through a Netlify serverless function..."
}
```

Response:

```
{
  "next_question": "You mentioned serverless functions — what are the cold start...",
  "score": {
    "score": 7,
    "depth": "strong",
    "relevance": "high",
    "feedback": "Good instinct on the proxy — next time mention the trade-offs...",
    "follow_up_needed": false
  },
  "rolling_average": 6.5,
  "difficulty_level": "medium",
  "difficulty_changed": false,
  "difficulty_direction": null,
  "turn_count": 1,
  "session_complete": false
}
```


GET /scorecard/{session_id}

Generate and return the final interview scorecard. Requires at least 2 completed turns.


```
{
  "overall_score": 7.2,
  "strengths": ["..."],
  "improvement_areas": ["..."],
  "hiring_recommendation": "yes",
  "per_turn_scores": [...],
  "summary": "...",
  "session_metadata": {
    "job_role": "Junior Frontend Engineer",
    "company_context": "Early-stage startup",
    "total_turns": 5,
    "final_difficulty_reached": "hard"
  },
  "conversation_history": [...],
  "raw_scores": [...]
}
```


9. The Adaptive Interview Loop

The single `POST /answer` endpoint *is* the adaptive engine. Each call executes, in order:

1. Validate the session exists and has required context.
2. Sanitize and append the candidate's answer to conversation history.
3. Run the EvaluatorAgent to score the answer → `AnswerScore`.
4. Store the score in the session.
5. Recalculate the rolling average across all turns.
6. Map the average to a difficulty level.
7. Run the InterviewerAgent to generate the next question with full updated context.
8. Append the next question to conversation history.

9. Return the next question, score, rolling average, and difficulty to the frontend.

Difficulty thresholds (`session_store.py`): rolling average `> 7` → `hard`, `< 4` → `easy`, otherwise `medium`. The interview is flagged complete at `turn_count >= 5`.

10. Agents & Prompts

Agent	Temperature	Input	Output
EvaluatorAgent	0.0	Question, answer, job role	`AnswerScore` JSON (score, depth, relevance, feedback, follow_up_needed)
InterviewerAgent	0.7	Resume, role, history, difficulty, follow-up signal	Next question (plain text)
Scorecard Generator	0.0	Full transcript, per-turn scores, resume	`FinalScorecard` JSON (holistic evaluation)

The evaluator's rubric scores 1–10 across three bands (shallow / adequate / strong) and returns a single actionable coaching sentence. The interviewer is explicitly instructed to never repeat a covered topic, always ground questions in the resume, and adapt tone to the difficulty level.

> **Defensive parsing.** Despite instructions to return raw JSON, Claude frequently wraps output in markdown fences. All agent responses are cleaned (fences stripped) before Pydantic validation; a parse failure raises `ValueError`, which the API surfaces as HTTP 500.

11. Data Models

AnswerScore

```
class AnswerScore(BaseModel):
    score: int                    # 1–10
    depth: Literal["shallow", "adequate", "strong"]
    relevance: Literal["low", "medium", "high"]
    feedback: str                 # one coaching line shown to candidate
    follow_up_needed: bool        # true if answer was incomplete
```

FinalScorecard

```
class FinalScorecard(BaseModel):
    overall_score: float
    strengths: List[str]
    improvement_areas: List[str]
    hiring_recommendation: Literal["strong_yes", "yes", "maybe", "no"]
    per_turn_scores: List[AnswerScore]
    summary: str
```

12. Testing

Backend unit tests

The backend uses Python's built-in `unittest` — no extra dependencies and no API key required. `pdfplumber` is mocked, so no real PDF files or network access are needed.

```
cd backend
source venv/bin/activate

# Run a single test module
python -m unittest test_resume_parser -v
```

> **Note:** `test_agent_loop.py` is **not** a unit test — it is a standalone terminal script that makes real calls to the Anthropic API and requires a valid key. Because it imports `agents.py` at module load, a blanket `python -m unittest discover` will attempt to import it and fail without a key. Run unit tests by module name instead.

Frontend checks

```
cd frontend
npm run lint     # ESLint
npm run build    # production build check
```

There is currently no frontend test runner (no Jest/Vitest setup).

13. Design Notes & Trade-offs

- **In-memory session store.** A plain module-level dict — fast and zero-infrastructure, but a server restart loses all sessions. The store is deliberately isolated so a Redis swap with TTL-keyed sessions is a one-file change.
- **Separate scorecard LLM call.** The final evaluation is not a mathematical average of turn scores — trajectory and recovery matter. A candidate who scored 3,3,9,9,9 differs fundamentally from one who scored 6,6,6,6,6, even with similar averages.
- **Agentic loop design.** Agents are treated as functions with specific inputs and outputs, composed deliberately — not as opaque "AI magic." Getting session-state injection right is the core engineering challenge.
- **Answer sanitization.** Candidate answers are Unicode-normalized and stripped of control characters before being appended to history, guarding JSON serialization against copy-pasted or voice-transcribed text.
